

```

@file:Suppress("DEPRECATION")
@file:OptIn(ExperimentalMaterial3Api::class)

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.platform.LocalContext

import com.arcgismaps.ArcGISEnvironment
import com.arcgismaps.data.*
import com.arcgismaps.geometry.*
import com.arcgismaps.mapping.*
import com.arcgismaps.mapping.layers.*
import com.arcgismaps.mapping.view.geometryeditor.GeometryEditor
import com.arcgismaps.toolkit.geoviewcompose.MapView
import kotlin.coroutines.flow.toList

import kotlin.coroutines.launch

@Composable
fun GeoTiffMapScreen() {

    val context = LocalContext.current
    ArcGISEnvironment.applicationContext = context.applicationContext

    val geometryEditor = remember { GeometryEditor() }
    val scope = rememberCoroutineScope()

    /* ----- Feature Tables ----- */

    val pointTable = remember {
        FeatureCollectionTable(
            listOf(Field(FieldType.Text, "name", "Name", 50)),
            GeometryType.Point,
            SpatialReference.wgs84()
        )
    }

    val lineTable = remember {
        FeatureCollectionTable(
            listOf(Field(FieldType.Text, "name", "Name", 50)),
            GeometryType.Polyline,
            SpatialReference.wgs84()
        )
    }
}

```

```

val polygonTable = remember {
    FeatureCollectionTable(
        listOf(Field(FieldType.Text, "name", "Name", 50)),
        GeometryType.Polygon,
        SpatialReference.wgs84()
    )
}

/* ----- Feature Collection Layer ----- */

val featureCollectionLayer = remember {
    FeatureCollectionLayer(
        FeatureCollection().apply {
            tables.add(pointTable)
            tables.add(lineTable)
            tables.add(polygonTable)
        }
    )
}

/* ----- Map ----- */

val map = remember {
    ArcGISMap(BasemapStyle.ArcGISLightGray).apply {
        operationalLayers.add(featureCollectionLayer)
        initialViewpoint = Viewpoint(
            Point(71.117965, 27.383511, SpatialReference.wgs84()),
            20000.0
        )
    }
}

/* ----- UI State ----- */

var showPoints by remember { mutableStateOf(true) }
var showLines by remember { mutableStateOf(true) }
var showPolygons by remember { mutableStateOf(true) }

/* ----- Layer Visibility ----- */

LaunchedEffect(showPoints, showLines, showPolygons) {

    val layers = featureCollectionLayer.layers.toList()

    if (layers.size >= 3) {

        val pointLayer = layers[0] as Layer
        val lineLayer = layers[1] as Layer
        val polygonLayer = layers[2] as Layer
    }
}

```

```

        pointLayer.isVisible = showPoints
        lineLayer.isVisible = showLines
        polygonLayer.isVisible = showPolygons
    }
}

/* ----- UI ----- */

Column(Modifier.fillMaxSize()) {

    MapView(
        modifier = Modifier.weight(1f),
        arcGISMap = map,
        geometryEditor = geometryEditor
    )

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(12.dp)
    ) {

        Text("Layer Visibility", style = MaterialTheme.typography.titleMedium)

        Row {
            Checkbox(checked = showPoints, onCheckedChange = { showPoints = it
it })
            Text("Points")
        }
        Row {
            Checkbox(checked = showLines, onCheckedChange = { showLines = it })
            Text("Lines")
        }
        Row {
            Checkbox(checked = showPolygons, onCheckedChange = { showPolygons =
it })
            Text("Polygons")
        }

        Spacer(Modifier.height(12.dp))

        Text("Drawing Mode", style = MaterialTheme.typography.titleMedium)

        Row(
            Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceEvenly

```

```

) {
    Button(onClick = { geometryEditor.start(GeometryType.Point) }) {
        Text("Point")
    }
    Button(onClick = { geometryEditor.start(GeometryType.Polyline) }) {
        Text("Line")
    }
    Button(onClick = { geometryEditor.start(GeometryType.Polygon) }) {
        Text("Polygon")
    }
}

```

```

Spacer(Modifier.height(8.dp))

```

```

Button(
    modifier = Modifier.fillMaxWidth(),
    onClick = {
        val geometry = geometryEditor.stop()

        geometry?.let {
            scope.launch {
                when (it) {
                    is Point -> {
                        val feature = pointTable.createFeature()
                        feature.geometry = it
                        feature.attributes.set("name", "Point")
                        pointTable.addFeature(feature)
                    }

                    is Polyline -> {
                        val feature = lineTable.createFeature()
                        feature.geometry = it
                        feature.attributes.set("name", "Line")
                        lineTable.addFeature(feature)
                    }

                    is Polygon -> {
                        val feature = polygonTable.createFeature()
                        feature.geometry = it
                        feature.attributes.set("name", "Polygon")
                        polygonTable.addFeature(feature)
                    }

                    else -> {
                        // Required because Geometry is sealed
                    }
                }
            }
        }
    }
)

```

```

    ) {
        Text("Save Geometry")
    }

    Spacer(Modifier.height(4.dp))

    Button(
        modifier = Modifier.fillMaxWidth(),
        colors = ButtonDefaults.buttonColors(
            containerColor = MaterialTheme.colorScheme.error
        ),
        onClick = {
            geometryEditor.stop()
            geometryEditor.clearGeometry()
        }
    ) {
        Text("Cancel Drawing")
    }
}
}
}
}

```